



Database Management Systems

Winter 2026

PropertyDekho.in

A Centralized Real Estate Management System

Project Assignment 1 Report

Team Members:

Shashank Kumar – 2024521

Namish Batra – 2024369

Mehardeep Singh Bhalla – 2024346

PROJECT OVERVIEW

PropertyDekho.in is a next-generation real estate platform designed to eliminate the fragmentation in the property market. Currently, the sector suffers from unorganized data, where buyers must visit multiple offline brokers to verify listings.

Our system centralizes data from owners, agents, and builders into a single **Normalized Relational Database**. Beyond simple storage, we leverage **Machine Learning** and **Computer Vision** to automate verification and enhance property discovery.

UNIQUE SELLING PROPOSITIONS (USP)

- **24-Hour Broker SLA:** A strict logic trigger ensures that once a client inquires, the assigned broker must schedule a meeting within 24 hours. Failure to do so lowers the broker's ranking score in our database.
- **AI-Powered Document Verification (Computer Vision):** To tackle fraud, we employ Computer Vision (OCR) to automatically scan and validate property documents uploaded by owners. This acts as a first-level filter before manual admin approval.
- **ML-Based Price Prediction:** Instead of static pricing, our system uses a Machine Learning regression model to suggest "Fair Market Value" to buyers based on historical data of the locality and amenities.

PROBLEM & SOLUTION

Current Market Pain Points

- **Data Redundancy:** The same flat is often listed by five different agents with conflicting prices.
- **Manual Verification:** Buyers have no quick way to verify if a builder has legal clearance.
- **Inefficient Search:** Traditional filters cannot handle complex queries like "Find 3BHKs with a pool, near a metro station, under 5 Crores."

Our DBMS Solution

- **3NF Normalized Schema:** We split Builders, Locations, and Properties into distinct tables to eliminate data redundancy.
- **Automated Integrity:** Database constraints ensure that a property cannot be listed with a negative price or invalid location.

- **Role-Based Security:** Sensitive data (like Owner contact info) is only accessible to verified Agents, enforced via View-level permissions.

SYSTEM ARCHITECTURE

User Role	Responsibilities
Admin	Manage Master Data (Cities, Amenities), Approve Agents, Audit Logs.
Owner/Broker	List properties (INSERT), Update status (UPDATE), Reply to inquiries.
Buyer/Tenant	Search listings (SELECT), Send Interest Requests, View verified docs.

DATABASE DESIGN

Key Entities

1. **Property:** Stores physical details (Area, BHK, Price, LocationID).
2. **User:** A single table for Admin, Agent, and Buyer (differentiated by RoleID).
3. **Inquiry:** Logs the interaction between a Buyer and a Property.

Key Relationships

- **1:N (One-to-Many):** One Builder constructs many Properties.
- **M:N (Many-to-Many):** Properties can have many Amenities (Gym, Pool), and an Amenity can exist in many Properties. Handled via a mapping table.

TECHNICAL IMPLEMENTATION

- **Database Core:** MySQL (Relational storage, Stored Procedures, Triggers)
- **Backend AI:** Python Flask + **Scikit-Learn** (Price Prediction) + **OpenCV** (Doc Verification)
- **Frontend:** HTML5, CSS3, JavaScript